



Spring MVC

1

Points communs pour JSP/Servlet/Rest

Spring MVC

2

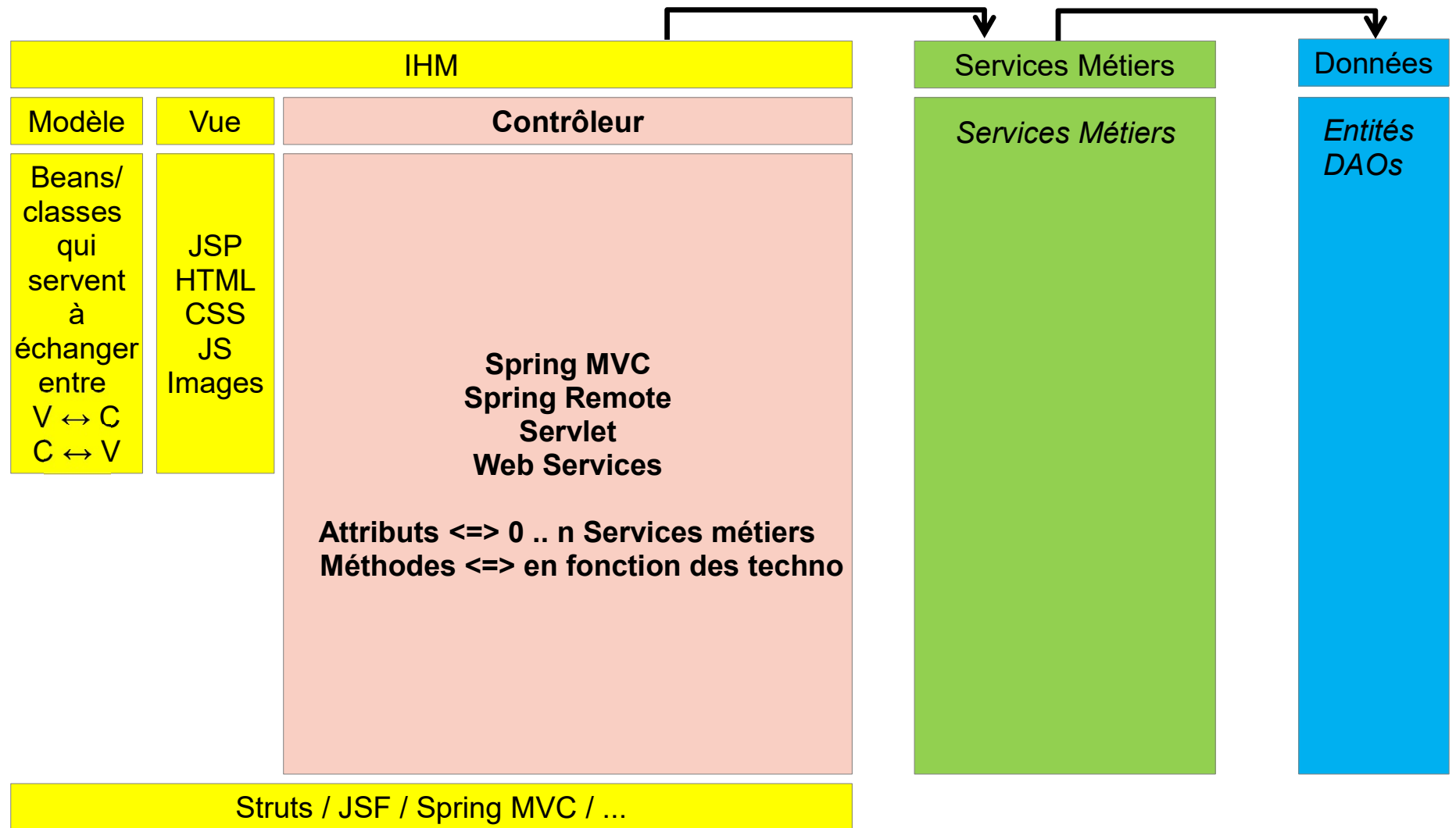
- ✓ Présentation
- ✓ Annotations Spring MVC

Présentation

- Spring dispose d'un framework web intégrant :
 - Le Pattern MVC2
 - Une technologie de binding et de validation
 - Une mécanique pour cacher les éléments JEE classiques (Session, Request, ...)

- Le Spring MVC regroupe les deux approches
 - Web Services Rest
 - JSP / Servlet

Présentation



Config Spring dans une application Web

Par fichier XML



➤ La configuration s'effectue dans le fichier web.xml

- Vous devez indiquer où sont vos fichiers Spring en ajoutant un context-param du nom de **contextConfigLocation** dans votre fichier **web.xml**

```
<context-param>  
  <param-name>contextConfigLocation</param-name>  
  <param-value>classpath*:spring/*-context.xml</param-value>  
</context-param>
```

❑ On parle ici des fichiers propres aux couches 'métiers' (@Repository, @Service)

- Vous devez aussi déclarer un listener qui va se charger de monter le/les fichiers de configuration Spring en mémoire

```
<listener>  
  <listener-class>org.springframework.web.context.ContextLoaderListener</listener-class>  
</listener>
```

Config Spring dans une application Web

- En mode web, vous ne devez **surtout pas** créer un `ClassPathXmlApplicationContext` pour le charger.
- Si vous avez besoin d'un bean dans vos contrôleurs, utilisez `@Autowired`
 - N'oubliez pas de déclarer le bean et de préciser le `component-scan` adéquat

Config Spring dans une application Web

Par fichier XML



- La configuration du Spring MVC se complètera avec (toujours dans le fichier **web.xml**)
 - La déclaration de la servlet principale du Spring

❑ Exemple :

```
<servlet>
  <servlet-name>dispatcher</servlet-name>
  <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
  <load-on-startup>1</load-on-startup>
</servlet>
```

- ❑ Rappel : load-on-startup force le conteneur JEE à charger la servlet au démarrage du serveur (et pas lors de sa première utilisation)

Config Spring dans une application Web Par fichier XML



- Le DispatcherServlet aura besoin d'un fichier de configuration Spring dédié
- Par défaut, utilisation de `/WEB-INF/${nom-servlet}-servlet.xml`
 - Ici, le fichier utilisé sera `/WEB-INF/dispatcher-servlet.xml`
- On parle ici du fichier de configuration des @Controller

Config Spring dans une application Web

Par fichier XML



- Vous pouvez surcharger la localisation du fichier de configuration
 - Via un paramètre d'initialisation (init-param)

```
<servlet>
  <servlet-name>dispatcher</servlet-name>
  <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
  <init-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>/WEB-INF/my-spring-servlet.xml</param-value>
  </init-param>
  <load-on-startup>1</load-on-startup>
</servlet>
```

Config Spring dans une application Web Par fichier XML



- Après la déclaration du DispatcherServlet, il faut spécifier son mapping
 - Les urls utilisées doivent suivre un mapping commun
 - Tous les mappings sont possibles.
 - Ex : *.smvc
 - Nous sommes toujours dans le fichier web.xml

```
<servlet-mapping>  
  <servlet-name>dispatcher</servlet-name>  
  <url-pattern>*.smvc</url-pattern>  
</servlet-mapping>
```

ou

```
<servlet-mapping>  
  <servlet-name>dispatcher</servlet-name>  
  <url-pattern>/rest/*</url-pattern>  
</servlet-mapping>
```

Config Spring dans une application Web

Par fichier XML



- Le fichier de configuration Spring dédié au Dispatcher (`/WEB-INF/${nom-servlet}-servlet.xml`) est un fichier de paramétrage XML comme les autres

- Il se chargera des contrôleurs et du paramétrage du Spring MVC

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:p="http://www.springframework.org/schema/p"
  xmlns:context="http://www.springframework.org/schema/context"
  xmlns:mvc="http://www.springframework.org/schema/mvc"
  xsi:schemaLocation="http://www.springframework.org/schema/beans http://www.springframework.org/schema/beans/spring-beans.xsd
    http://www.springframework.org/schema/context http://www.springframework.org/schema/context/spring-context.xsd
    http://www.springframework.org/schema/mvc http://www.springframework.org/schema/mvc/spring-mvc.xsd">

  <context:component-scan base-package="com.controleur" />

  <mvc:annotation-driven />

  <bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
    <!-- ... -->
  </bean>

  <bean id="messageSource" class="org.springframework.context.support.ReloadableResourceBundleMessageSource">
    <!-- ... -->
  </bean>
</beans>
```

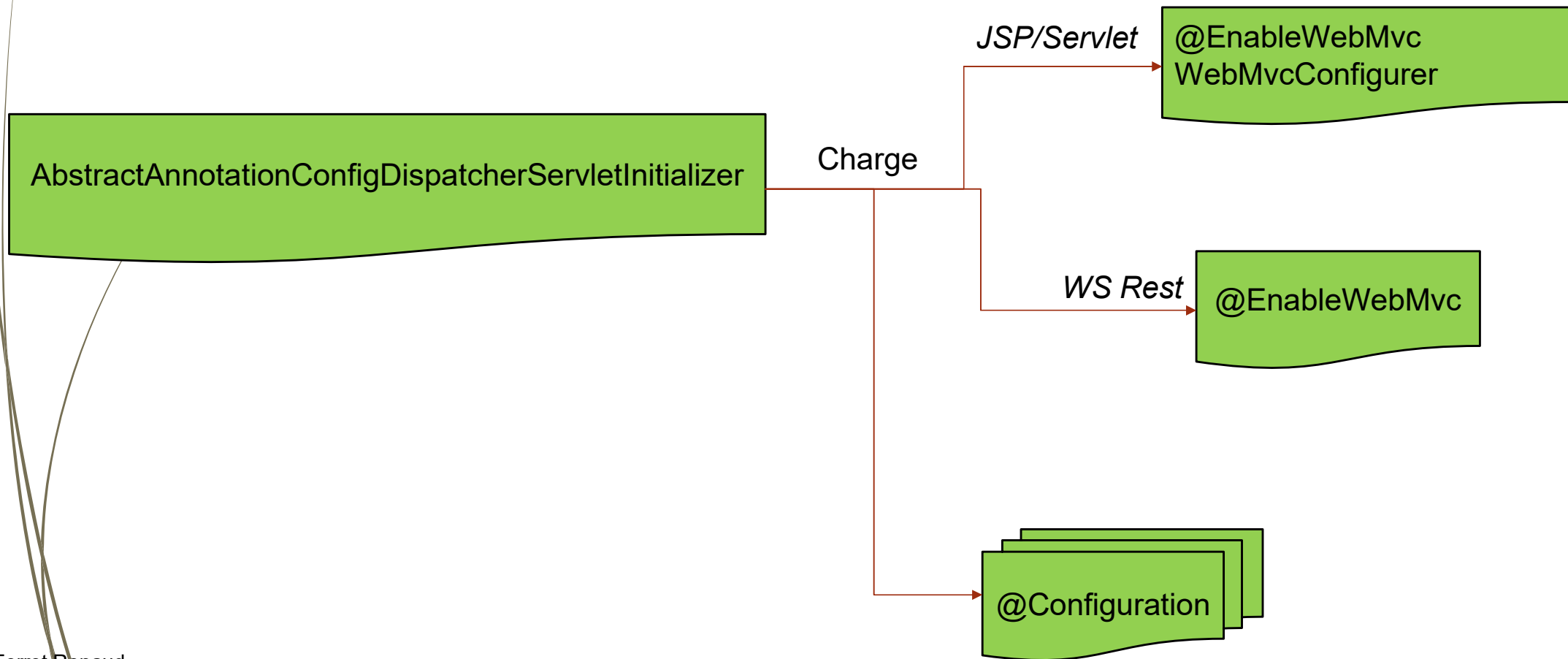
Config Spring dans une application Web

Par classes Java



- Il y aura au moins deux classes Java à écrire
 - La première se chargera de configurer la partie Web de l'application
 - ❑ Elle héritera d'une classe spéciale qui implémente l'interface **WebApplicationInitializer** (il faut être en JEE 3.0 minimum)
 - Par exemple **AbstractAnnotationConfigDispatcherServletInitializer**
 - La seconde s'occupera du paramétrage de Spring MVC
 - ❑ Dans tous les cas elle sera annotée en `@Configuration` et `@EnableWebMvc`
 - ❑ Dans le cas des JSP/Servlet : Elle implémentera l'interface **WebMvcConfigurer**
- Les autres classes seront des classes de configuration standards, annotées en `@Configuration`

Config Spring dans une application Web Par classes Java



Config Spring dans une application Web

Par classes Java



➤ Charge les autres classes de configurations

➤ Porte le mapping JEE (les extensions URL) associé aux actions du Spring

```
public class MonInitializer extends AbstractAnnotationConfigDispatcherServletInitializer {

    @Override
    protected Class<?>[] getRootConfigClasses() {
        return new Class[] { MaConfigurationAnnoteeSpring.class };
    }

    @Override
    protected Filter[] getServletFilters() {
        return new Filter[] { new FormContentFilter() };
    }

    @Override
    protected Class<?>[] getServletConfigClasses() {
        return new Class[] { MaConfigurationSpringMVC.class };
    }

    @Override
    protected String[] getServletMappings() {
        return new String[] { "*.smvc" };
    }

}
```

Config Spring dans une application Web Par classes Java



➤ WebMvcConfigurerAdapter

- Est annotée **@EnableWebMvc** spécifique
- Est une classe **@Configuration** spécifique : s'occupe des @Controller via son @ComponentScan
- Peut, selon son héritage, porter l'ensemble des paramétrages/Beans du Spring MVC
 - ❑ Chargement de resolver, des fichiers internationalisés, ...

```
@Configuration
@EnableWebMvc
@ComponentScan(basePackages = "com.controleur")
public class MaConfigurationSpringMVC implements WebMvcConfigurer {

    @Bean
    public ViewResolver viewResolver() { ... }

    @Bean
    public ReloadableResourceBundleMessageSource messageSource() { ... }

    @Override
    public void addResourceHandlers(ResourceHandlerRegistry pRegistry) { ... }
}
```

Config Spring dans une application Web

Par classes Java



➤ Les autres fichiers de @Configuration

- Portent le paramétrage de l'application, côté métier (@Repository, @Service)
- Portent le paramétrage de l'application, côté technique (mail, Ldap, DataSource, ...)

```
@Configuration
@PropertySource("classpath:spring/database.properties")
@ComponentScan({ "com.dao", "com.service" })
@EnableTransactionManagement
public class MaConfigurationAnnoteeSpring {

    @Bean
    public DataSource dataSource() throws NamingException {
        // ...
    }

    @Bean
    public LocalSessionFactoryBean sessionFactory(DataSource dataSource, ...) {
        // ...
    }

    @Bean
    public HibernateTransactionManager transactionManager(SessionFactory sessionFactory) {
        // ...
    }
}
```


Annotations du Spring MVC - @Controller

- Le contrôleur (en annotations)
 - Pas besoin d'héritage de classe/interface spécifique
 - Pas besoin de surcharger une méthode spécifique
 - Chaque méthode est une action disponible sur un URL

```
@Controller  
@RequestMapping("/simple")  
public class SimpleController {  
  
    @GetMapping  
    public String faireQQchose(HttpServletRequest request, HttpServletResponse response) throws Exception {  
        ...  
    }  
  
    @PostMapping  
    public String faireAutreChose() throws Exception {  
        ...  
    }  
}
```

Liens entre @Controller et @Service

- Les @Service sont injectés dans les @Controller par le Spring
- Rappels :
 - Un contrôleur **n'est pas** un service métier
 - Les services métiers portent le transactionnel
 - Un contrôleur ne doit pas être lié à un @Repository

```
@Controller
@RequestMapping("/login")
public class LoginController {
    @Autowired
    private final IClientService clientService;
    ...
}
```

Exemple :
Injection via
l'attribut

Annotations du Spring MVC - @RestController

- Le contrôleur associé aux web service Rest
 - Est lui-même un @Controller (et un @ResponseBody)
 - Disponible à partir de Spring 4

```
@RestController
@RequestMapping("/simple")
public class SimpleRestController {

    @GetMapping
    public String faireQQchose(HttpServletRequest request, HttpServletResponse response) throws Exception {
        ...
    }

    @PostMapping
    public String faireAutreChose() throws Exception {
        ...
    }
}
```

Spring MVC - @RequestMapping

- Annotation de classe et/ou de méthode
- Permet d'indiquer l'URL de votre action
- Sur une classe : Indique la racine de l'URL
 - ❑ @RequestMapping("/comptes")
- Sur une méthode : s'additionne au chemin de la classe
 - ❑ @RequestMapping(value = "/lister" ...)
 - ❑ /comptes/lister

Spring MVC - @RequestMapping

➤ Exemple de déclaration

```
@Controller
@RequestMapping("/login")
public class LoginController {

    @Autowired
    private IAuthenticationService service;

    @RequestMapping("/dolt")
    public String doLogin(...) {
    }
}
```

➤ Ce contrôleur sera joignable sur l'URL

- ❑ `http(s)://[server]:[port]/[contextjee]/[urlmappingspring]/[requestmappingclass]/[requestmappingmethode]`
- ❑ `http://localhost:8080/netbank/login/dolt`

Spring MVC - @RequestMapping

- **@RequestMapping** permet aussi de définir
 - les variables d'URL : informations qui se trouvent sur l'URL
 - ❑ `@RequestMapping(value = "/maj/{id}" ...`
 - le type de données en entrées et en sorties
 - ❑ `@RequestMapping(produces = MediaType.APPLICATION_JSON_VALUE, consumes = ...)`
 - les commandes HTTP supportées (get, put, ...)
 - ❑ `@RequestMapping(method = { RequestMethod.GET, RequestMethod.PUT })`

Spring MVC - @RequestMapping

➤ Exemple :

```
@Controller
@RequestMapping("/login")
public class LoginController {

    @Autowired
    private IAuthenticationService service;

    @RequestMapping(value = "/dolt", method = RequestMethod.POST)
    public String doLogin(...) {
    }
}
```

➤ Ce contrôleur sera joignable sur l'URL

❑ <http://localhost:8080/netbank/login/dolt> en **POST** uniquement

Spring MVC - @RequestMapping

- Depuis Spring v4.3 vous pouvez remplacer l'annotation `@RequestMapping` sur vos méthodes par :
 - `@GetMapping`
 - `@PostMapping`
 - `@PutMapping`
 - `@DeleteMapping`
 - `@PatchMapping`
- Evite d'avoir à indiquer *method =*

Spring MVC - @XxxMapping

- Les annotations @XxxMapping supportent le filtrage conditionnel en fonction

- D'un paramètre

```
@GetMapping(path = "/pets", params = "pNom=pValeur")  
public void findPet() {  
    // ...  
}
```

- ☐ L'action vérifie avant de s'exécuter qu'il y a un paramètre pNom avec la valeur pValeur

- D'un header

```
@GetMapping(path = "/pets", headers = "hClef=hValeur")  
public void findPet() {  
    // ...  
}
```

- ☐ L'action vérifie avant de s'exécuter qu'il y a un header hClef avec la valeur hValeur

Spring MVC - @RequestMapping

- Depuis Spring v5 vous pouvez utiliser des patterns complexes pour vos URL
 - `/pages/t?st.html` : ? représente un caractère
 - `/resources/*.png` : * représente n caractères
 - `/resources/**` : tous les chemins/fichiers sous `/resources/` (ex: `/resources/image.png` et `/resources/css/spring.css` ...)
 - `/resources/{*path}` : Tous les chemins/fichiers sous `/resources/` et place dans un paramètre `path` le chemin (`/resources/image.png` alors `path = "/image.png"`, `/resources/css/spring.css` alors `path = "/css/spring.css"`)
- <https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/web/util/pattern/PathPattern.html>

Spring MVC - @RequestParam

- Annotation sur les paramètres de la méthode
 - ... faireQQC(**@RequestParam** String login, ...)
- Permet de récupérer un paramètre provenant d'un formulaire ou d'un URL
- Le typage du paramètre n'a pas obligation d'être une chaîne de caractères
- Vous devez indiquer le *name* du paramètre dans l'annotation
 - C'est le *name* dans le formulaire ou la clef dans votre URL
 - Remarque : en spring < 4, il n'y a pas d'attribut name pour cette annotation, le nom du paramètre Java DOIT être le name du parameter.
- Vous pouvez ajouter l'option *required* dans l'annotation pour indiquer le fait que le paramètre est obligatoire

Spring MVC - @RequestParam

- C'est Spring qui s'occupera de gérer la conversion si le typage n'est pas une String
- Si vous indiquez comme typage un objet à vous, et que le consumes indique JSON ou Xml, c'est aussi le Spring qui se chargera de faire la transformation
 - JAX-B pour l'XML
 - Jackson pour le Json où 'nom attribut' = 'clef json'
 - ☐ Attention aux majuscules / minuscules
- C'est aussi Spring qui va gérer les erreurs :
 - code 400 (Bad Request) si le paramètre est absent et en *required=true*

Spring MVC - @RequestParam

➤ Exemple :

```
105 @RequestMapping(value = "/byparam",  
106                 method = { RequestMethod.GET, RequestMethod.PUT, RequestMethod.POST },  
107                 produces = MediaType.APPLICATION_JSON_VALUE)  
108 public @ResponseBody ResponseEntity<String> authentifierByParam(  
109     @RequestParam(name = "login", required = true) String login,  
110     @RequestParam(name = "password", required = true) String password) {  
111
```

- Login et password sont
 - ☐ des paramètres classiques
 - ☐ de type String
 - ☐ obligatoires

Spring MVC - @RequestAttribute

- Depuis Spring 4.3+, plutôt que de faire apparaître l'HttpRequest comme paramètre de votre méthode vous pouvez faire usage de l'annotation **@RequestAttribute**

- C'est une annotation de paramètre

```
@RequestMapping("/faire")  
public void helloWorld(@RequestAttribute("clef") User aUser) {  
    // ...  
}
```

Fait par le Spring

```
User aUser = (User) request.getAttribute("clef")
```

Spring MVC - @ResponseBody

- Annotation sur le retour de la méthode
 - `public @ResponseBody ResponseEntity<String> faireQQC(...)`
- Indique que le retour de la méthode doit être lié au corps de la réponse HTTP
 - Peut être un objet de type `ResponseEntity<T>`
 - Peut être une `String`
 - Peut être un `Object` à vous (simple ex: `Compte`, ou composé `List<Compte>`)
- Dans le cas où Jackson est présent et que le retour doit être en JSON
 - c'est le Spring qui s'occupera de gérer les transformations `Object -> JSon`

Spring MVC - @ResponseBody

- Si vous êtes en Spring 4+
 - L'annotation peut se positionner sur la classe @Controller et s'applique à toutes ses méthodes (plus besoin de la mettre sur chaque méthode)
- Si vous êtes sur une classe @RestController
 - Pas besoin de la positionner sur les méthodes, elle est déjà incluse dans @RestController

Spring MVC - @ResponseBody

➤ Exemple :

```
105 @RequestMapping(value = "/byparam",  
106                 method = { RequestMethod.GET, RequestMethod.PUT, RequestMethod.POST },  
107                 produces = MediaType.APPLICATION_JSON_VALUE)  
108 public @ResponseBody ResponseEntity<String> authentifierByParam(  
109     @RequestParam(name = "login", required = true) String login,  
110     @RequestParam(name = "password", required = true) String password) {  
111
```

➤ Ici la réponse sera un objet ResponseEntity

- ❑ Un body
- ❑ Un HTTP Status

Spring MVC - @CrossOrigin

- Cette annotation a été mise en place depuis Spring 4.2
- Elle permet de gérer les appels venant d'une autre origine que celle du site JEE
- Elle peut se placer sur la classe ou sur la méthode d'un @Controller
- @CrossOrigin(origins = "http://localhost:9000")

Spring MVC - @CrossOrigin

- Si vous souhaitez centraliser la gestion du CORS (Cross-origin resource sharing), vous pouvez le faire via un @Bean WebMvcConfigurerAdapter

```
@Bean
public WebMvcConfigurer corsConfigurer() {
    // Exemple en Spring < 5
    return new WebMvcConfigurerAdapter() {
        @Override
        public void addCorsMappings(CorsRegistry registry) {
            registry.addMapping("/**").allowedMethods("**").allowCredentials(true).allowedHeaders("**");
        }
    };
}
```

Spring MVC - @CrossOrigin

- En **Spring 5+** on utilisera l'interface **WebMvcConfigurer** (à la place de **WebMvcConfigurerAdapter**)

```
@Bean
public WebMvcConfigurer corsConfigurer() {
    // Exemple en Spring >= 5
    return new WebMvcConfigurer() {
        @Override
        public void addCorsMappings(CorsRegistry registry) {
            registry.addMapping("/**").allowedMethods("*").allowCredentials(true).allowedHeaders("*");
        }
    };
}
```

Spring MVC - @CrossOrigin

- Autre approche (mais toujours avec le `WebMvcConfigurer`)

@Configuration

```
public class CorsConfig implements WebMvcConfigurer {
```

@Override

```
public void addCorsMappings(CorsRegistry registry) {  
    registry.addMapping("/**")  
        .allowedOrigins("https://example.com")  
        .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")  
        .allowedHeaders("*")  
        .maxAge(3600);  
}  
}
```

Spring MVC - @CrossOrigin

- Dans le cas où vous faites usage de **Spring Security**, c'est lui qui gèrera le CrossOrigin
 - Il faudra donc le faire **exclusivement** dans **sa** classe/fichier de configuration

Spring MVC - @PathVariable

- Annotation sur les paramètres de la méthode
 - ... faireQQC(**@PathVariable** String smp, ...)
- Fonctionne avec @RequestMapping (ou @GetMapping, @XxxMapping) quand l'URL est composée d'informations dynamiques (avec des {})
 - @RequestMapping(value = "/maj/{**smp**}") ...

Spring MVC - @PathVariable

➤ Exemple :

```
@GetMapping("/byurl/{login}/{password}")  
public ResponseEntity<String> authentifierByUrl(  
    @PathVariable String login,  
    @PathVariable String password) {  
    // ...  
}
```

➤ Le contrôleur sera joignable uniquement en GET sur l'URL

❑ <http://.../byurl/df/df>

Spring MVC - @RequestBody

- **@RequestBody** est une annotation qui se place sur les paramètres des méthodes d'un @Controller
- Elle indique au Spring que l'élément est à récupérer du body de la requête. On l'utilise souvent dans le cas où le client envoie ses données en JSON ou XML.
- Le typage du paramètre
 - **String** : vous gérez le flux JSON vous-même dans votre méthode
 - **Object** : Spring va faire la transformation [Object ↔ JSON/XML] pour vous
- @RequestBody **n'est JAMAIS utilisable avec un GET**

Spring MVC - @RequestBody

- La classe LoginBean
 - Objet bête ⇔ attribut + get/set
 - Le typage doit respecter les contraintes JSon (pas de Date ou alors il faudra 's'arranger')
- Le flux JSon (entrée ou sortie)
 - {
 - "login":"df",
 - "password":"df"
- Attention : clef = getClef / setClef
 - Respectez scrupuleusement les majuscules minuscules

```

13 public class LoginBean implements Serializable {
14
15     private static final long serialVersionUID = 1L;
16
17     private String login;
18     private String password;
19
20     /**
21      * Constructeur de l'objet.
22      */
23     public LoginBean() {
24         super();
25     }
26
27     * Recupere la valeur du login.
28     public String getLogin() {}
29
30     * Recupere la valeur du password.
31     public String getPassword() {}
32
33     * Modifie la valeur du login.
34     public void setLogin(String aValue) {}
35
36     * Modifie la valeur du password.
37     public void setPassword(String aValue) {}
38
39     public String toString() {}
40 }

```

Spring MVC - @RequestBody

➤ Exemple :

- La classe LoginBean sert vraiment de modèle au sens du MVC
⇔ Objet d'échange entre la vue et le contrôleur.

```
58 @RequestMapping(method = {RequestMethod.POST },  
59                 produces = MediaType.APPLICATION_JSON_VALUE,  
60                 consumes = MediaType.APPLICATION_JSON_VALUE)  
61 public @ResponseBody ResponseEntity<String> authentifierPost(@RequestBody LoginBean loginBean) {
```

- Le Spring, via Jackson, se chargera de transformer le flux JSON d'entrée en objet LoginBean
 - Idem pour le flux de sortie
- Rappel : @RequestBody **n'est pas utilisable avec un GET**

Spring MVC - Scopes

- En JEE vous avez des objets standards
 - **ServletContext** (scope application)
 - **HttpSession** (scope session)
 - **HttpServletRequest** (scope request)
 - **HttpResponse**
- Si vous en avez besoin, ajoutez simplement comme paramètre un élément de type *HttpSession*, *HttpServletRequest*, *HttpResponse*
 - Le Spring se chargera de vous les donner (il fera l'injection automatiquement)
- Attention ! : Vous ne pouvez pas faire de même avec **ServletContext**

Gestion du Scope Application

- Pour accéder au scope de l'application, vous devez ajouter un objet de type ***ServletContext*** dans votre contrôleur
 - Et lui indiquer qu'il est en **@Autowired**
 - ☐ Votre attribut sera géré par le Spring et vous donnera accès au scope application
- Ou, ne pas mettre **@Autowired** et implémenter l'interface `org.springframework.web.context.ServletContextAware`

Gestion du Scope Request

➤ Pour accéder à l'objet **HttpRequest**

- Vous pouvez simplement ajouter un attribut de type *HttpServletRequest* à vos méthodes

 - ☐ Spring injectera la request automatiquement

- Ou, dans les cas qui ne permettent pas d'ajouter des paramètres, vous pouvez faire usage

 - `RequestContextHolder.currentRequestAttributes()`**

```
HttpServletRequest request = ((ServletRequestAttributes)  
RequestContextHolder.currentRequestAttributes()).getRequest();
```

Gestion du Scope Request

- Pour faire usage du `RequestContextHolder.currentRequestAttributes()`
 - Il faut être une classe qui hérite de `org.springframework.web.context.request.RequestContextListener` (*pas obligatoire en Spring Boot*)
 - Il faut être déclaré en tant que **listener** auprès du conteneur JEE (*pas obligatoire en Spring Boot*)

Gestion du scope session

- En Spring 4.3+, plutôt que d'exposer l'objet *HttpSession*, pour une méthode, vous pouvez utiliser l'annotation de paramètre **@SessionAttribute** (sans 's')

```
@RequestMapping("/faire")  
public void helloWorld(@SessionAttribute("clef") User aUser) {  
    // ...  
}
```

Fait par le Spring

```
User aUser = (User) request.getSession(true).getAttribute("clef")
```


Gestion des *Cookies*

- Pour une méthode, vous pouvez utiliser l'annotation de paramètre **@CookieValue** afin de récupérer la valeur contenue dans un cookie

```
@RequestMapping("/faire")  
public void helloWorld(@CookieValue("clef") String aValue) {  
    // ...  
}
```

Fait par le Spring

```
Cookie[] lesCookies = request.getCookies();  
String aValue = null;  
for (Cookie unCookie : lesCookies) {  
    if ("clef".equals(unCookie.getName())) {  
        aValue = unCookie.getValue();  
    }  
}
```

Gestion du *Header*

- Pour une méthode, vous pouvez utiliser l'annotation de paramètre **@RequestHeader** afin de récupérer la valeur contenue dans une information du Header

```
@RequestMapping("/faire")  
public void helloWorld(@RequestHeader("clef") String aValue) {  
    // ...  
}
```

Fait par le Spring

```
String aValue = request.getHeader("clef");
```